

Name: - Shubham Das
Date: - 19/09/2026

S. A. P I. D: - 590033500
Programme: - B. TECH (CSE)

Assignment: - Write a C programme that displays the following menu and uses a switch statement to print the 'Right-aligned patterns' selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate parameter-less function with no return value.

Note: - For each activity, prepare the algorithm and test-case table. Then write, compile and execute the C programme, and test it against the prepared test cases.

1. Right-aligned increasing triangle: - Input the number of rows, n, and print a right-aligned increasing triangle using asterisks. For n = 4:

```

          *
        * *
      * * *
    * * * *
```

2. Right-aligned increasing number triangle: - Input the number of rows, n, and print a right-aligned triangle of natural numbers. In each row, print the natural numbers. In each row, print the natural numbers from 1 up to the corresponding row number. For n = 4:

```

          1
        1 2
      1 2 3
    1 2 3 4
```

3. Right-aligned decreasing triangle: - Input the number of rows, n, and print a right-aligned decreasing triangle using Asterisks. For n = 4:

```

    * * * *
      * * *
        * *
          *
```

Part 1: - Thorough understanding of the problem statement

Input:- *choice* (Menu option selected by the user, Variable type: *int*, Unit: unitless integer)

n (Number of rows for the selected pattern, Variable type: *int*, Unit: unitless integer)

Output: - Right-aligned increasing asterisk triangle pattern

Right-aligned increasing natural number triangle pattern

Right-aligned decreasing asterisk triangle pattern

Formula: - *Option 1 (Right-aligned increasing asterisk triangle):*

Spaces per row *i* : $n - 1$

Asterisks per row *i* : *i*

Nested loop logic: Outer loop *i* from 1 to *n*; space loop *j* from 1 to $n - 1$; character loop *k* from 1 to *i*.

Option 2 (Right-aligned increasing number triangle):

Spaces per row *i* : $n - i$

Numbers per row: *i* : 1 up to *i*

Nested loop logic: Outer loop *i* from 1 to *n*; space loop *j* from 1 to $n - 1$; number loop *k* from 1 to *i* printing *k*.

Option 3 (Right-aligned decreasing asterisk triangle):

Spaces per row: *i* : $i - 1$

Asterisks per row: *i* : $n - i + 1$

Nested loop logic: Outer loop *i* from 1 to *n*; space loop *j* from 1 to $i - 1$; character loop *k* from 1 to $n - i + 1$

Part 2: - Writing, compiling and evaluating the algorithm

Step 1: - Start

Step 2: - Declare functions with *void* return type and parameterless arguments:

void rightAlignedIncreasingTriangle(void),

void rightAlignedNumberTriangle(void),

void rightAlignedDecreasingTriangle(void).

Step 3: - Inside *main()*, declare integer variable *choice*.

Step 4: - Execute a *do-while* loop to display the pattern options menu until the user chooses to exit:

1. Right-aligned increasing triangle

2. Right-aligned increasing number triangle

3. Right-aligned decreasing triangle

4. Exit

Step 5: - Read *choice* and evaluate it using a *switch* statement.

Step 6 (Option 1 Algorithm): - Read row count *n*. If $n \leq 0$, display an error.

Otherwise, for *i* from 1 to *n*;

Print $n - i$ spaces.

Print *i* asterisks (*).

Print newline ($\backslash n$).

Step 7 (Option 2 Algorithm): - Read row count *n*. If $n \leq 0$, display an error.

Otherwise, for *i* from 1 to *n*:

Print $n - 1$ spaces.

For *k* from 1 to *i*, spaces.

Print newline ($\backslash n$).

Step 8 (Option 3 Algorithm): - Read row count *n*. If $n \leq 0$, display an error.

Otherwise, for *i* from 1 to *n*:

Print $i - 1$ space.

Print $n - i + 1$ asterisks (*).

Print newline ($\backslash n$).

Step 9 (Default Case): - Display an "Invalid choice" error message.

Step 10: Stop when option 4 (Exit) is selected.

Part 3: - Writing and compiling the C programme

```

GNU nano 9.0
#include <stdio.h>

// Parameterless function prototypes with void return type
void rightAlignedIncreasingTriangle(void);
void rightAlignedNumberTriangle(void);
void rightAlignedDecreasingTriangle(void);

int main() {
    int choice;

    do {
        printf("\n=====\\n");
        printf("                Right-Aligned Pattern Menu                \\n");
        printf("=====\\n");
        printf("1. Right-aligned increasing triangle\\n");
        printf("2. Right-aligned increasing number triangle\\n");
        printf("3. Right-aligned decreasing triangle\\n");
        printf("4. Exit\\n");
        printf("Enter your choice (1-4): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                rightAlignedIncreasingTriangle();
                break;
            case 2:
                rightAlignedNumberTriangle();
                break;
            case 3:
                rightAlignedDecreasingTriangle();
                break;
            case 4:
                printf("Exiting program. Goodbye!\\n");
                break;
            default:
                printf("Error: Invalid choice. Please select options 1-4.\\n");
        }
    } while (choice != 4);

    return 0;
}

// 1. Right-aligned increasing triangle using asterisks
void rightAlignedIncreasingTriangle(void) {
    int n, i, j, k;

    printf("\\nEnter the number of rows (n): ");
    scanf("%d", &n);

```

```

GNU nano 9.0
printf("\nEnter the number of rows (n): ");
scanf("%d", &n);

if (n <= 0) {
    printf("Error: Number of rows must be greater than zero.\n");
    return;
}

printf("\n--- Output Pattern ---\n");
for (i = 1; i <= n; i++) {
    // Print leading spaces
    for (j = 1; j <= n - i; j++) {
        printf(" ");
    }
    // Print asterisks
    for (k = 1; k <= i; k++) {
        printf("*");
    }
    printf("\n");
}
}

// 2. Right-aligned increasing number triangle
void rightAlignedNumberTriangle(void) {
    int n, i, j, k;

    printf("\nEnter the number of rows (n): ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Error: Number of rows must be greater than zero.\n");
        return;
    }

    printf("\n--- Output Pattern ---\n");
    for (i = 1; i <= n; i++) {
        // Print leading spaces
        for (j = 1; j <= n - i; j++) {
            printf(" ");
        }
        // Print natural numbers from 1 to i
        for (k = 1; k <= i; k++) {
            printf("%d", k);
        }
        printf("\n");
    }
}
}

```

GNU nano 9.0

```
printf("\nEnter the number of rows (n): ");
scanf("%d", &n);

if (n <= 0) {
    printf("Error: Number of rows must be greater than zero.\n");
    return;
}

printf("\n--- Output Pattern ---\n");
for (i = 1; i <= n; i++) {
    // Print leading spaces
    for (j = 1; j <= n - i; j++) {
        printf(" ");
    }
    // Print natural numbers from 1 to i
    for (k = 1; k <= i; k++) {
        printf("%d", k);
    }
    printf("\n");
}
}

// 3. Right-aligned decreasing triangle using asterisks
void rightAlignedDecreasingTriangle(void) {
    int n, i, j, k;

    printf("\nEnter the number of rows (n): ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Error: Number of rows must be greater than zero.\n");
        return;
    }

    printf("\n--- Output Pattern ---\n");
    for (i = 1; i <= n; i++) {
        // Print leading spaces
        for (j = 1; j < i; j++) {
            printf(" ");
        }
        // Print asterisks
        for (k = 1; k <= n - i + 1; k++) {
            printf("*");
        }
        printf("\n");
    }
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the terminal

Step 2: - Create the source code

Step 3: - Write and Save Code

Step 4: - Compile the programme

Step 5: - Run the Binary Executable Programme

```
(kali㉿kali)-[~]  
$ nano pattern_menu.c  
  
(kali㉿kali)-[~]  
$ gcc pattern_menu.c -o pattern_menu  
  
(kali㉿kali)-[~]  
$ ./pattern_menu
```

```
=====
```

Right-Aligned Pattern Menu

```
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 1

Enter the number of rows (n): 4

--- Output Pattern ---

```
  *  
  **  
 ***  
****
```

```
=====
```

Right-Aligned Pattern Menu

```
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 2

Enter the number of rows (n): 4

--- Output Pattern ---

```
  1  
  12  
 123  
1234
```

```
=====
```

Right-Aligned Pattern Menu

```
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 3

```
=====
                        Right-Aligned Pattern Menu
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 3

Enter the number of rows (n): 4

--- Output Pattern ---

**

*

```
=====
                        Right-Aligned Pattern Menu
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 1

Enter the number of rows (n): 1

--- Output Pattern ---

*

```
=====
                        Right-Aligned Pattern Menu
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 2

Enter the number of rows (n): -3

Error: Number of rows must be greater than zero.

```
=====
                        Right-Aligned Pattern Menu
=====
```

1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit

Enter your choice (1-4): 9

```
=====
                        Right-Aligned Pattern Menu
=====
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice (1-4): 2

Enter the number of rows (n): -3
Error: Number of rows must be greater than zero.

=====
                        Right-Aligned Pattern Menu
=====
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice (1-4): 9
Error: Invalid choice. Please select options 1-4.

=====
                        Right-Aligned Pattern Menu
=====
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
4. Exit
Enter your choice (1-4): 4
Exiting program. Goodbye!

(kali@kali) - [~]
$ █
```

Part 5: - Standard Test Table

TEST CASE	SELECTED OPTION	INPUT (n)	EXPECTED OUTPUT PATTERN	ACTUAL OUTPUT PATTERN	RESULT (PASS/FAIL)
1	1	4	*	*	PASS

			** *** *****	** *** *****	
2	2	4	1 12 123 1234	1 12 123 1234	PASS
3	3	4	***** *** ** *	***** *** ** *	PASS

4	1	1	*	*	PASS
5	2	-3	Error: Number of rows must be greater than zero.	Error: Number of rows must be greater than zero.	PASS
6	9	N/A	Error: Invalid choice. Please select options 1-4.	Error: Invalid choice. Please select options 1-4.	PASS

Part 6: - Evaluation

The solution adheres strictly to all specified constraints:

1. Menu-driven design utilizing a *switch* statement.
2. Nested loops without arrays.
3. Modular, parameterless functions (*void function_name(void)*) with *void* return types.
4. Input validation for non-positive row numbers and out-of-range menu options.